

CS571 Programming Languages

Spring 2026

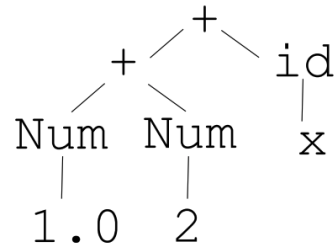
2. Syntax: lexing

Compiler Internal Structure

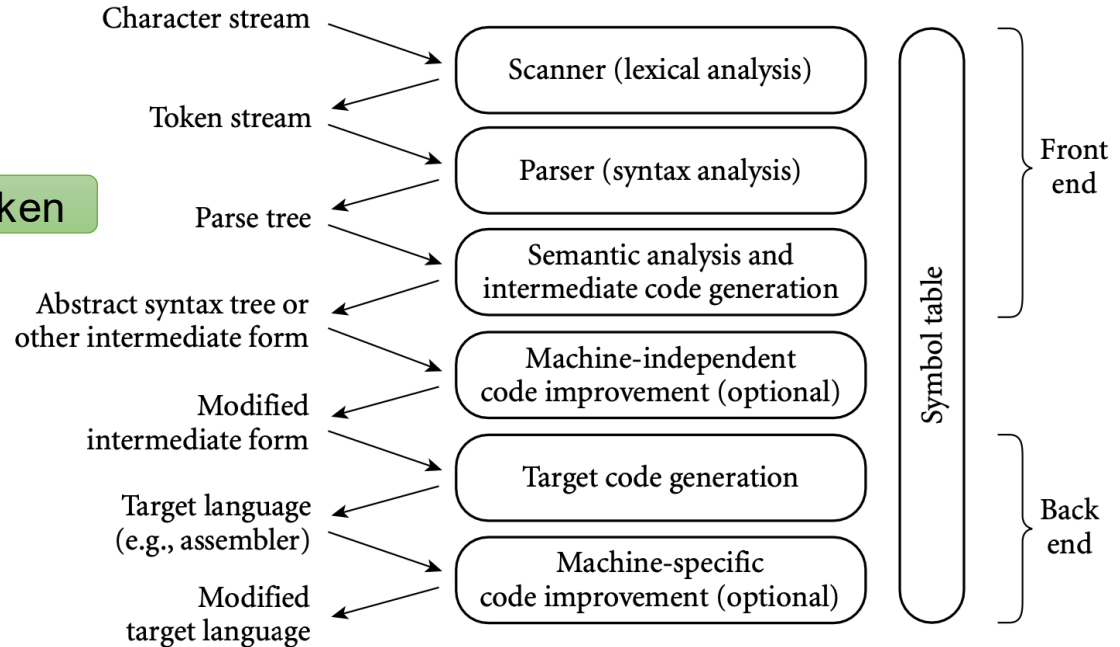
$(1.0 + 2) + x$

(1.0 + 2) + x

Token



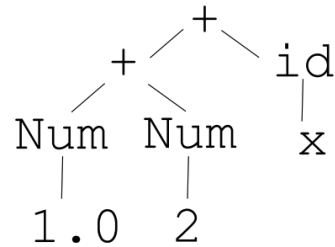
AST



Interpreter Internal Structure

$(1.0 + 2) + x$

(1.0 + 2) + x



AST

Token

Abstract syntax tree or
other intermediate form

Character stream

Token stream

Parse tree

Program input

Scanner (lexical analysis)

Parser (syntax analysis)

Semantic analysis and
intermediate code generation

Tree-walk routines

Symbol table

Front
end

Program output

Copyright © Elsevier Inc.

Syntax vs. Semantics

- Syntax: how a program is written

Scheme

C

(+ 1 2)

1 + 2

- Semantics: what's the meaning of a program

Sum of 1 and 2



Agenda

1. Theory of lexing: regular expressions
2. How to build a lexer: finite automata

Lexical Analysis (Scanning or Tokenizing)

- Identify the atoms: converts a stream of characters into a stream of **tokens**
 - **Keywords**: e.g. if and while
 - **Literals or constants**: e.g. 12, “hello”
 - **Special symbols**: e.g. “;”, “<=“
 - **Identifiers**: e.g. x, y, zz, myfun
- Discards white space, tabs and comments between the tokens
- Tag tokens with type and row/column number



Lexical Analysis Example

- Consider the following C GCD function:

```
/* Compute GCD of integers read from input */  
int gcd(int i, int j) {  
    while (i != j) {  
        if (i > j) {  
            i = i - j; }  
        else {  
            j = j - i;  
        }  
    } return i;  
}
```



Lexical Analysis Example

- Lexical analysis will take the character stream constituting the program and transform it into the following token stream:

```
int    gcd    (    int    i    int    j    )
{      while  (    i      !=    j      )    {
if     (      i      >      j      )    {    i
=      i      -      j      ;      }    else  {
j      =      j      -      i      ;      }
return i      ;    }
```

- Note the removal of whitespace and comments.



Designing a Lexical Analyzer (Lexer)

- It's more than just comparing a string is “int” or not: there are infinite integers, and you cannot enumerate all integers in your lexer!
- Turns out **Finite Automata (DFA/NFA)** handle the situation very well!

Formal Languages

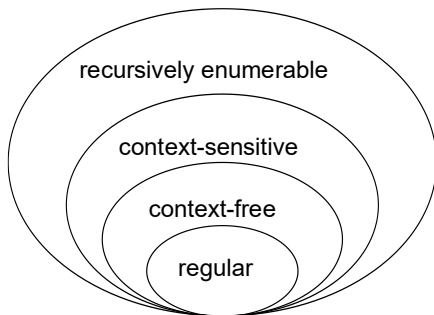
- A formal language is a **set of (legal) strings** from a **finite alphabet Σ (character set)**
 - Some languages (sets) are finite
 - IPv4 address: $\{0.0.0.0, \dots, 255.255.255.255\}$
 - Most languages are infinite
 - The set of all C programs
- Goal: a concise & precise notation for specifying a language

Chomsky Hierarchy

Noam Chomsky



Chomsky in 2017



Programming
languages



Grammar	Languages	Automaton
Type-0	Recursively enumerable	Turing machine
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine
Type-2	Context-free	Non-deterministic pushdown automaton
Type-3	Regular	Finite state automaton

Images from Wikipedia



Regular Expression

- **Regular expressions (regex, RE)** are the primary method for specifying the lexical aspects of a programming language
- Regular expressions are used to specify the sequence of characters which constitute the tokens of a programming language
- Regular expressions are also used in many tools like grep or programming languages like Java and Python for string pattern matching

RegEx Definition

A regex over a vocabulary Σ is defined inductively as follows:

- **Empty:** the empty regex ϵ defines an empty lang.
- **Symbol:** if $a \in \Sigma$, then a is a regex denoting the lang. with only “ a ”
- **Concatenation:** if A and B are regexes, then AB is a regex denoting the lang. containing concatenations of strings defined by A with strings defined by B
- **Alternation:** if A and B are regexes, then $A \mid B$ is a regex denoting the lang. containing strings defined either by A or by B

RegEx Definition

- **Kleene Closure:** if A is a regex, then A^* is a regex denoting the lang. containing concatenations of **0-or-more** strings defined by A
 - In other words, $A^* = \varepsilon \mid A \mid AA \mid AAA \mid \dots$
- Default **precedence:** closure > concatenation > alternation
 - Closure~power, concatenation~multiplication, alternation~addition
 - Parentheses can be used to override the default precedence
- Characters like $|$ and $*$ and $()$ are **metacharacters** in that they are part of the regex notation
 - They can be matched literally by using a preceding $\backslash$: e.g. $\backslash|$ and $\backslash*$

More Formally

Each RE R defines a language (set of strings) $L(R)$:

- A character x : $L(x) = \{ "x" \}$
- Empty string ϵ : $L(\epsilon) = \{ "" \}$
- Concatenation: $L(RS) = \{ r.s \mid r \in L(R), s \in L(S) \}$
- Alternation: $L(R|S) = L(R) \cup L(S)$
- Kleene star: $L(R^*) = \{ "" \} \cup L(R) \cup L(RR) \cup \dots$

. means string concatenation

Examples

RE	$L(RE)$
a	
ac	
$a c$	
a^*	
$(a b)(c d)$	
$(ab) (cd)$	
$((a b)c)^*$	

Examples

RE	$L(RE)$
a	$\{“a”\}$
ac	$\{“ac”\}$
$a c$	$\{“a”, “c”\}$
a^*	$\{“”, “a”, “aa”, “aaa”, \dots\}$
$(a b)(c d)$	$\{“ac”, “ad”, “bc”, “bd”\}$
$(ab) (cd)$	$\{“ab”, “cd”\}$
$((a b)c)^*$	$\{“”, “ac”, “bc”, “acac”, “acbc”, “bcbc”, \dots\}$

More Examples

RE	$L(RE)$
$ab cd$	
$a bc^*d$	

More Examples

RE	$L(RE)$
$ab cd$	
$(ab) (cd)$	$\{“ab”, “cd”\}$
$a bc^*d$	
$a (b(c^*)d)$	$\{“a”, “bd”, “bcd”, “bccd”, “bcccd”, \dots\}$

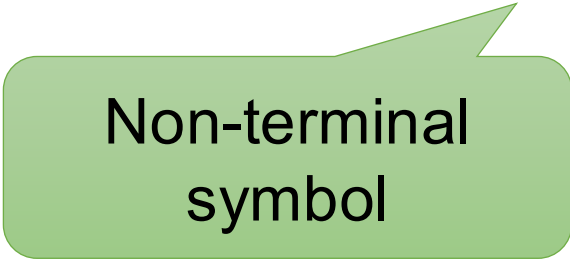
UNIX Extensions to RE

Extension	Core RE
$[abcd]$	$a b c d$
$.$ (wild cast)	$a b c \dots$ (all char. except new line)
$a?$	εa
a^+	aa^*
$[a - z]$	$a b c \dots z$
$[^ab]$	all char. except those in $[ab]$
$a\{3,5\}$	$aaa aaaa aaaaa$
$\backslash [\backslash] \backslash . \backslash d$	$[\] \ . \ [0-9]$

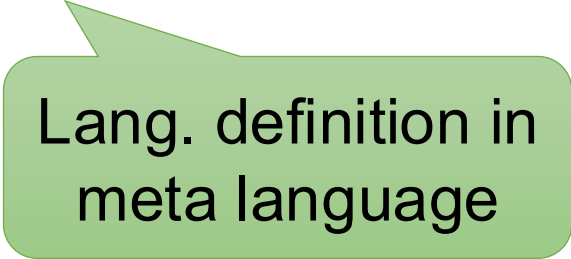
Grammar

- How to write down a RE grammar?

$$A \rightarrow \omega$$



Non-terminal
symbol



Lang. definition in
meta language

- Each such rule is called a production

Language of tokens (in C)

- Identifier:
 - Letters, digits and underscore '_' only
 - The first character must be an underscore or a letter
 - Identifier $\rightarrow$ [`_a-zA-Z`][`_a-zA-Z0-9`]*
- Number:
 - digits, decimal point, suffix such as "l", "u"
 - Number $\rightarrow$ (`0` | [`1-9`][`0-9`]*) (`\.` [`0-9`]+)? (`l`? | `u`?)

Language of tokens (in C)

- Operators: $+$, $-$, $*$, $/$, ...
 - Operator $\rightarrow + \mid - \mid * \mid / \mid \dots$
- Keywords : if, while, for, int, ...
 - Keyword $\rightarrow \text{if} \mid \text{while} \mid \text{for} \mid \text{int} \mid \dots$
- Punctuation: $\{$ $\}$ $[$ $]$ $;$...
 - Punctuation $\rightarrow \{ \mid \} \mid \backslash [\mid \backslash] \mid ; \mid \dots$

Correctness

- Is a RE grammar correct?
 - Does it cover all legal strings?
 - Does it allow any illegal string?

Examples

bit	?
4 bits	?
(non-empty) bits	?
(non-empty) even # of bits	?
Integers between 0 and 255	?
IPv4 address	?
bits with equal # of 0's and 1's	?

Examples

bit	0 1
4 bits	(0 1)(0 1)(0 1)(0 1)
(non-empty) bits	(0 1)+
(non-empty) even # of bits	((0 1)(0 1))+
Integers between 0 and 255	[0-9] [1-9][0-9] 1[0-9][0-9] 2[0-4][0-9] 25[0-5]
IPv4 address	Use RE above
bits with equal # of 0's and 1's	Cannot be expressed in RE



Agenda

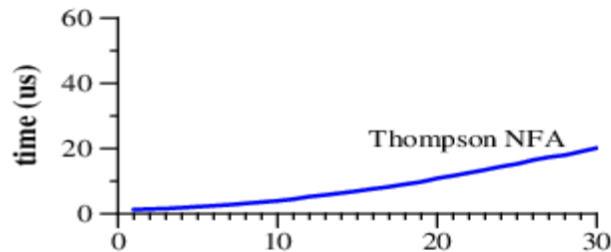
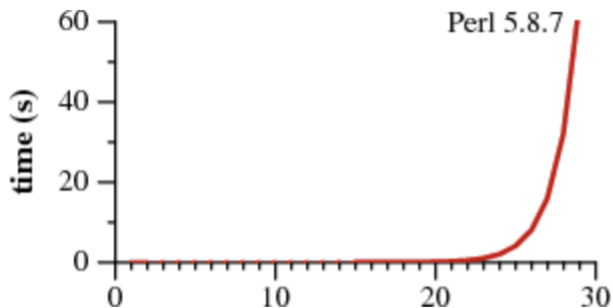
1. Theory of lexing: regular expressions
2. How to build a lexer: finite automata

Scanner Implementation

- Backtracking
 - To match the regex $A|B$
 - First, try to match A
 - If that doesn't work, backtrack and try to match B
- Finite Automata

RegEx vs. Backtracking

- Experiment: match the RegEx $a^n a^n$ on the string a^n
- E.g. for $n=3$, match $a? a? a? aaa$ on “ $aaaa$ ”



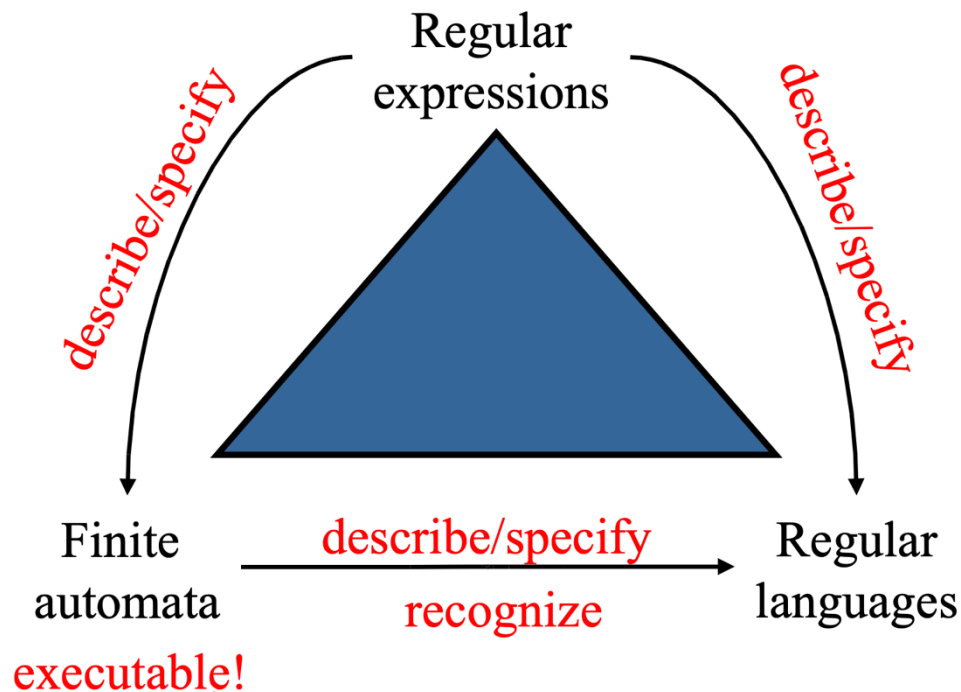
Time to match $a^n a^n$ against a^n

Credit: [Russ Cox](#)

RegEx vs. Finite Automata

- It turns out RegExes and the various automata formalisms are equivalent
- RegEx implementation strategy: compile to an automaton

RegEx vs. Scanner



RE: generator, generate strings in a regular language.

Scanner (DFAs): recognizer, given a string, says yes/no to whether the string is in a regular language.

Finite Automata (FA)

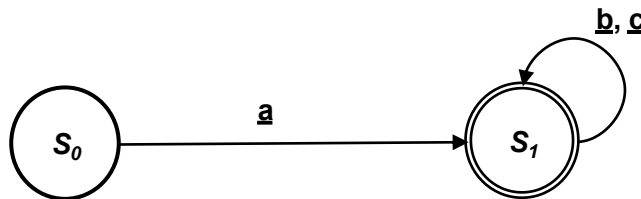
- A finite automaton (FA) is a tuple $A = \langle Q, \Sigma, T, S, F \rangle$
 - Q a finite non-empty set of **states**
 - Σ a finite **alphabet** of input letters
 - $T \subseteq Q \times \Sigma \times Q$ a set of **transitions**
 - $S \subseteq Q$ a set of **start states**
 - $F \subseteq Q$ a set of **final (accepting) states**

Finite Automata (FA)

- FA accepts a string $\omega = \omega_0\omega_1 \dots \omega_n \in \text{Seq}(\Sigma)$ if and only if there exists a sequence of states $s_0s_1 \dots s_{n+1}$ such that
 - s_0 is the **start state**
 - s_{n+1} is an **accepting state**
 - For all $i = 0 \dots n$, $(s_i, \omega_i, s_{i+1}) \in T$ is a **transition**
- FAs are pictorially represented as directed graphs, commonly known as state diagrams or transition diagrams

Finite Automata: Example

a (b | c)^{*}



- $Q = \{s_0, s_1\}$
- $\Sigma = \{a, b, c\}$
- $T = \{(s_0, a, s_1), (s_1, b, s_1), (s_1, c, s_1)\}$
- $S = \{s_0\}$
- $F = \{s_1\}$

Types of FA

- **Deterministic finite automata (DFA)**: only one transition allowed out of each (state, symbol) pair
- **Nondeterministic finite automata (NFA)**: multiple transitions out of each (state, symbol) pair
- **ϵ -transitions**: transitions that do not consume any of the input string



NFA Implementation

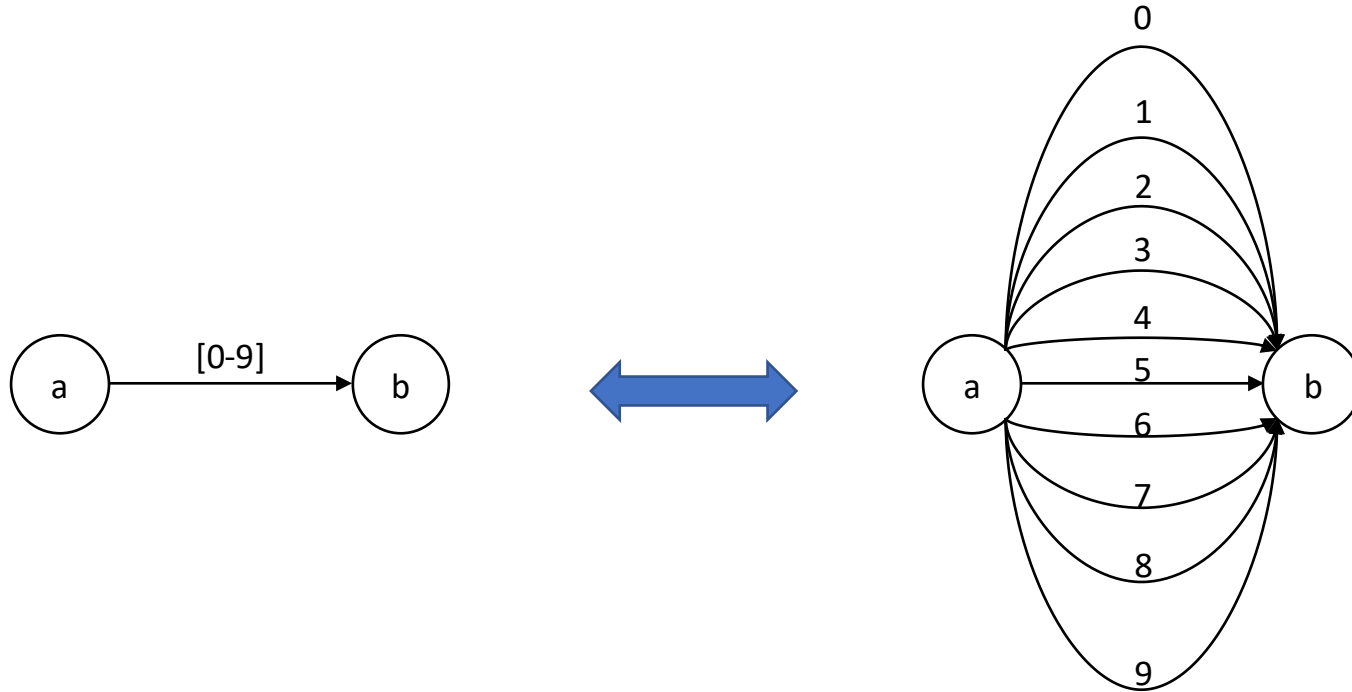
- To execute an NFA, we need to keep track of the *set* of current possible states.
- We will use this technique to implement an *NFA interpreter*



NFA Implementation Example

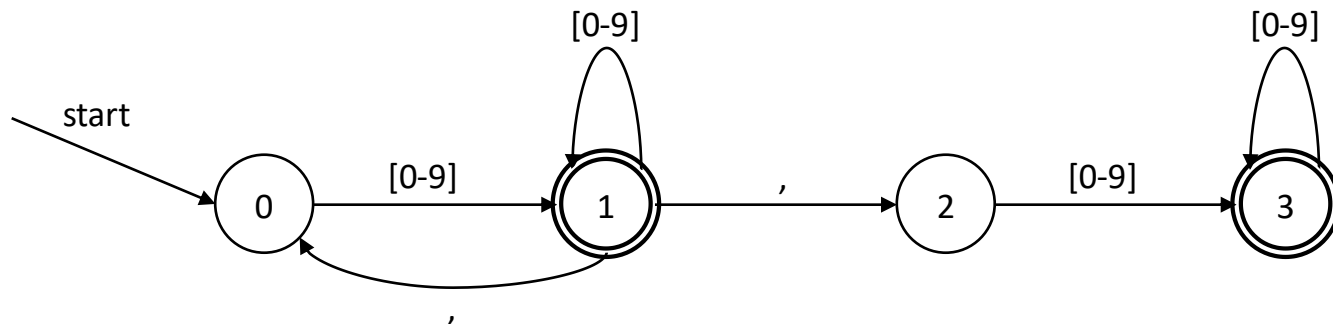
- RegEx: `([0-9]+,)*[0-9]+`
- String: 123,321,789

A Useful Notation



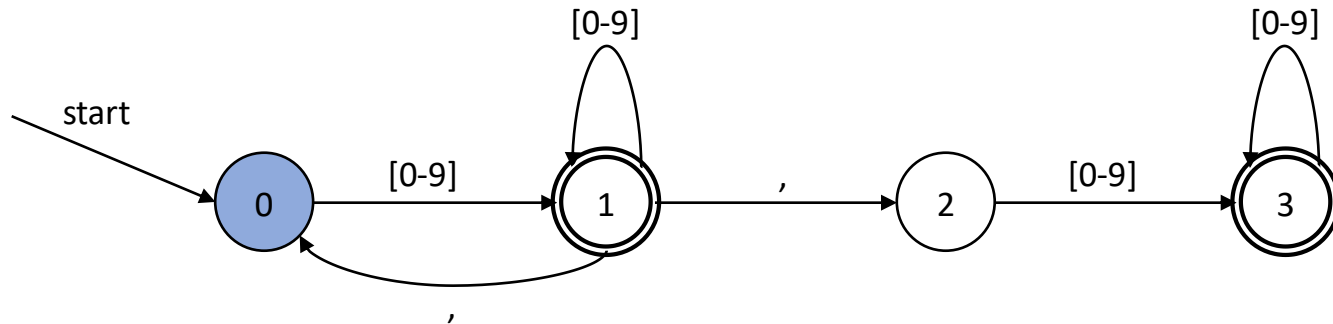
NFA Implementation Example

- RegEx: $([0-9]^+,)^*[0-9]^+$
- String: 123,321,789



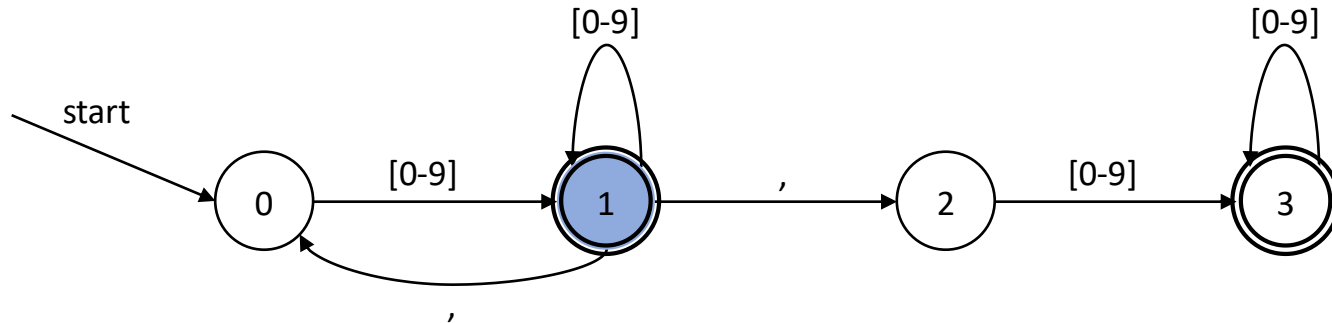
NFA Implementation Example

↓
1 2 3 , 3 2 1 , 7 8 9



NFA Implementation Example

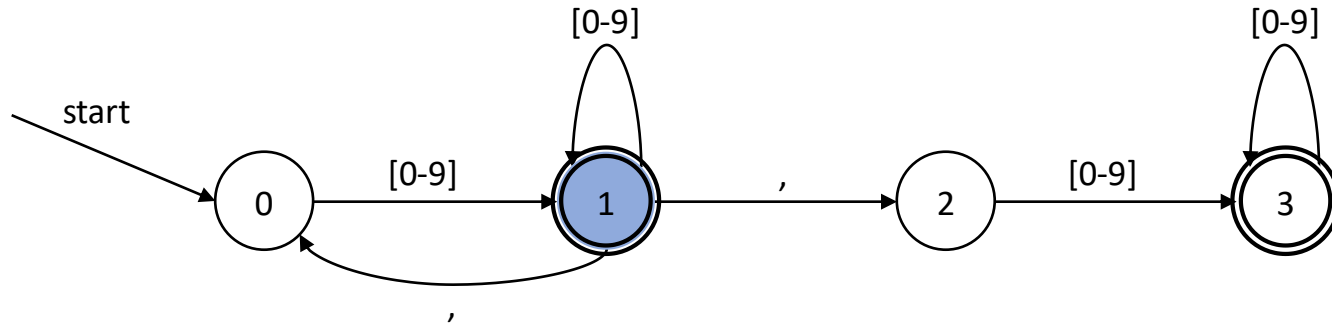
↓
1 2 3 , 3 2 1 , 7 8 9



NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9

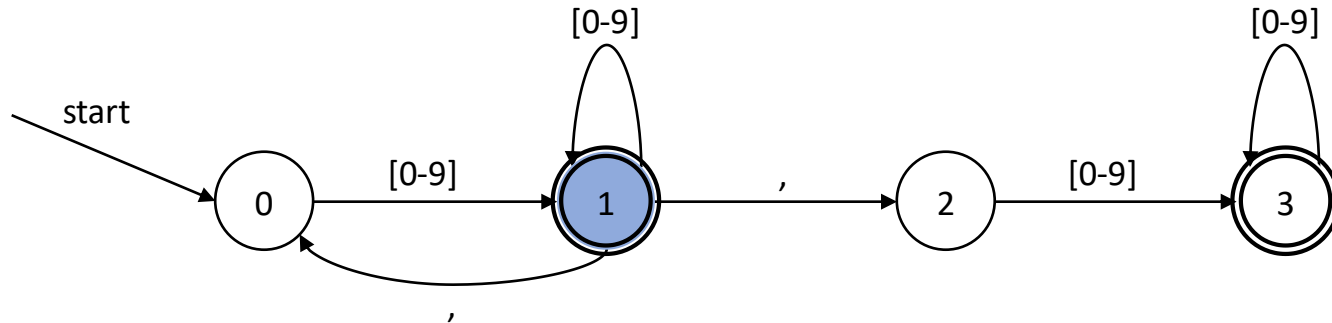
↓



NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9

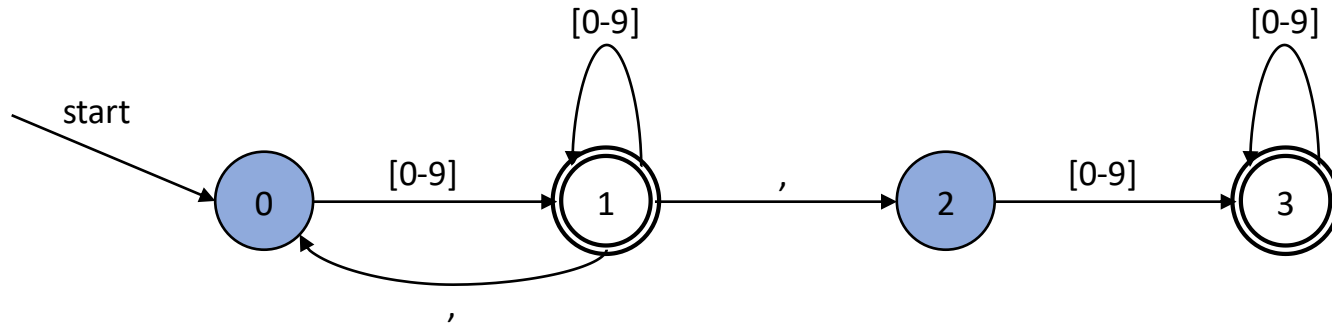
↓



NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9

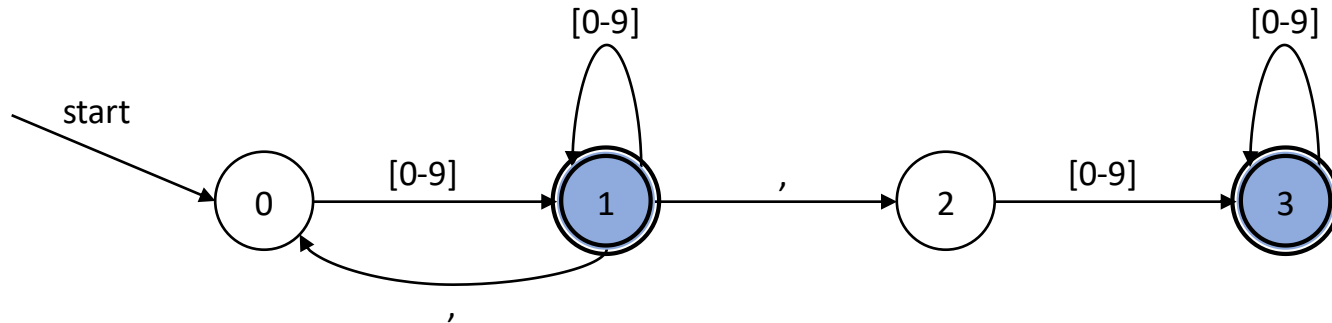
↓



NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9

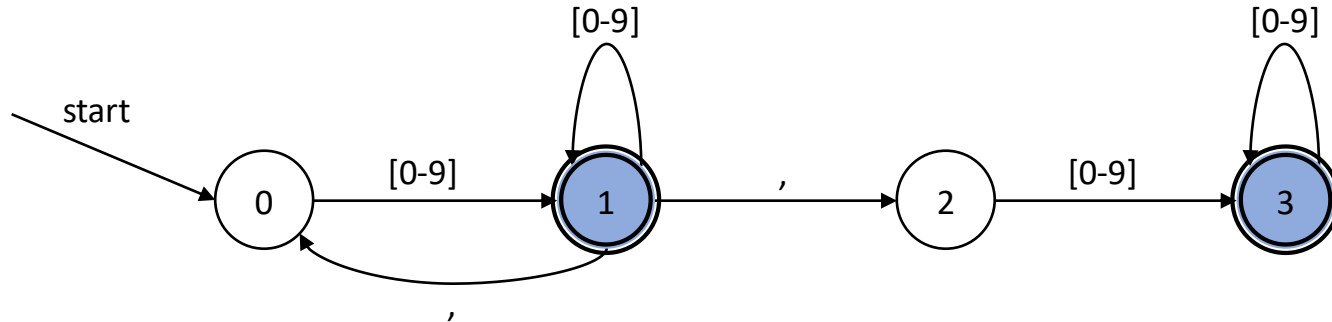
↓



NFA Implementation Example

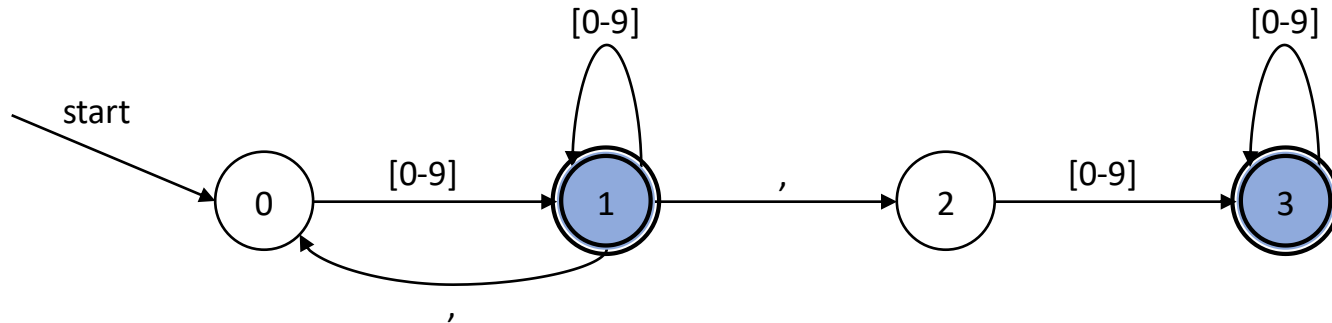
1 2 3 , 3 2 1 , 7 8 9

↓



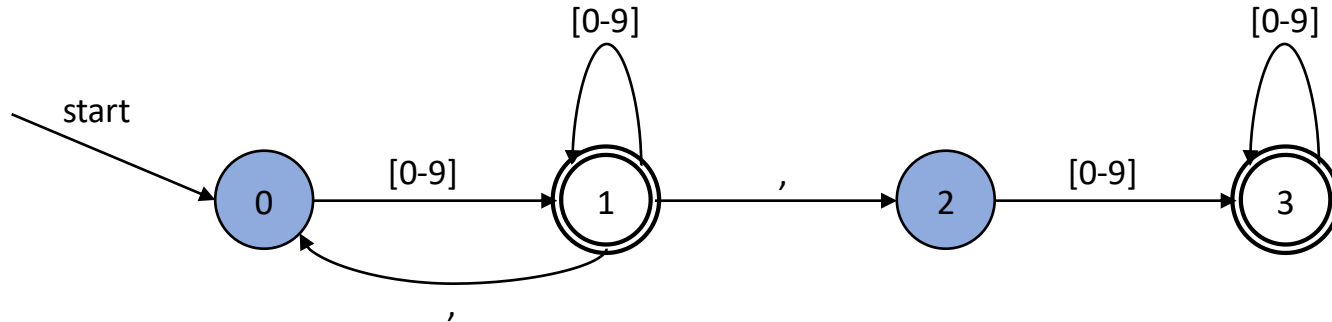
NFA Implementation Example

1 2 3 , 3 2 1 ↓ , 7 8 9



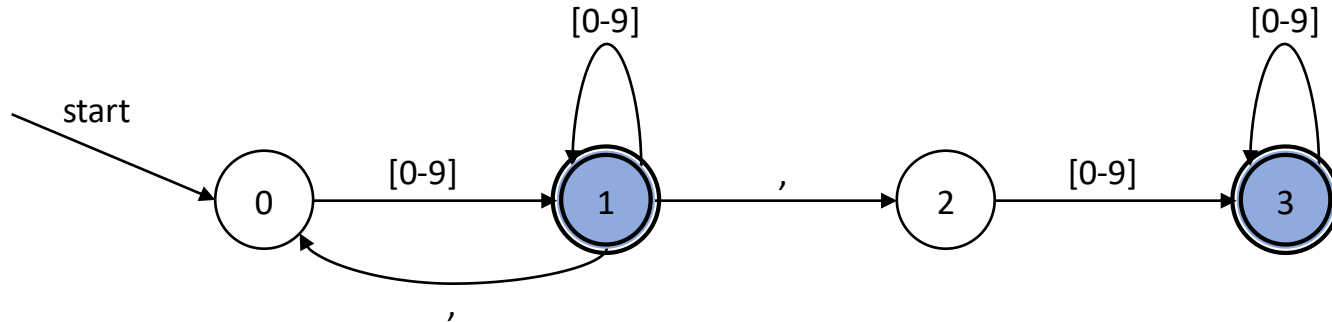
NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9



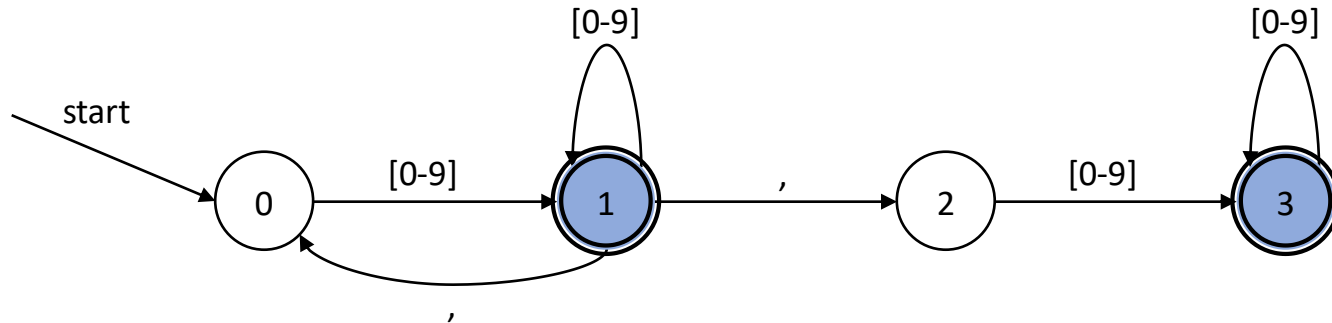
NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9



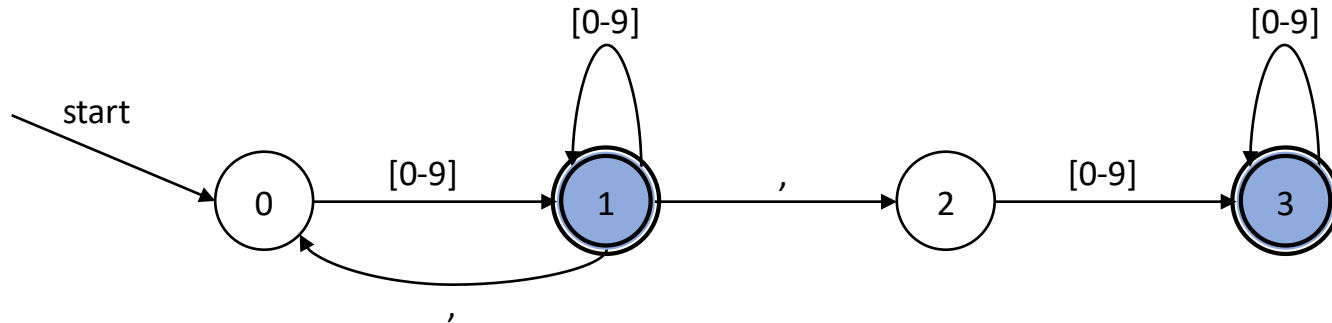
NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9

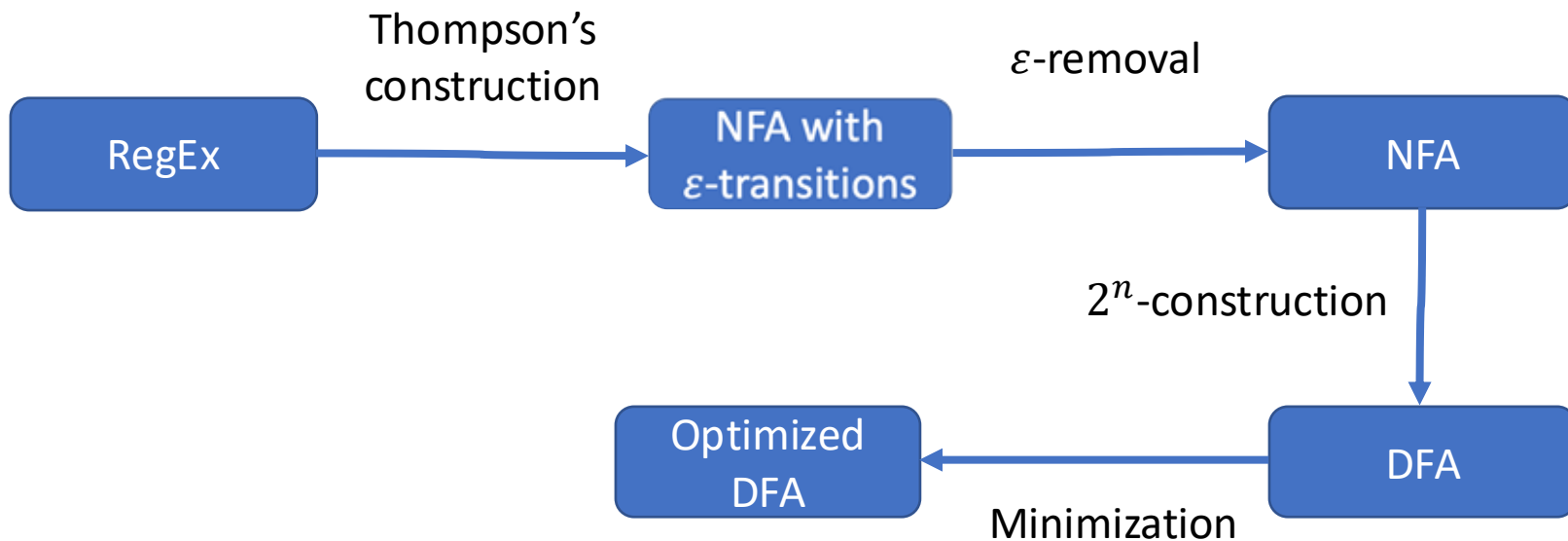


NFA Implementation Example

1 2 3 , 3 2 1 , 7 8 9 ↓



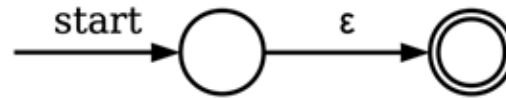
RegEx vs. Finite Automata



Thompson's Construction

- The Goal: convert a RegEx to an NFA with ε -transitions with more restrictions:
 - The NFA has **exactly one accepting state**
 - The NFA has **no transitions into its start state**
 - The NFA has **no transitions out of its accepting state**

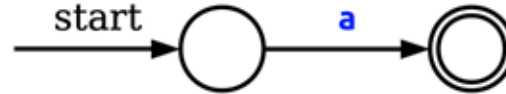
Rules for Base Case



Automaton for ϵ

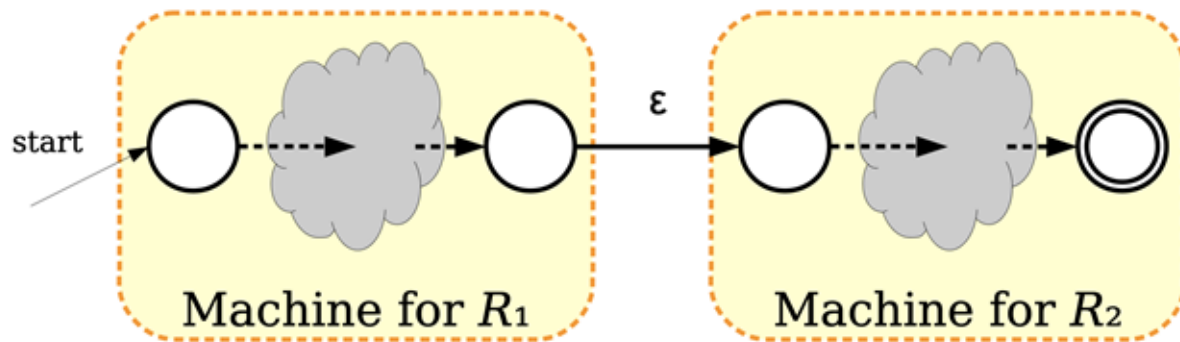


Automaton for $\emptyset$



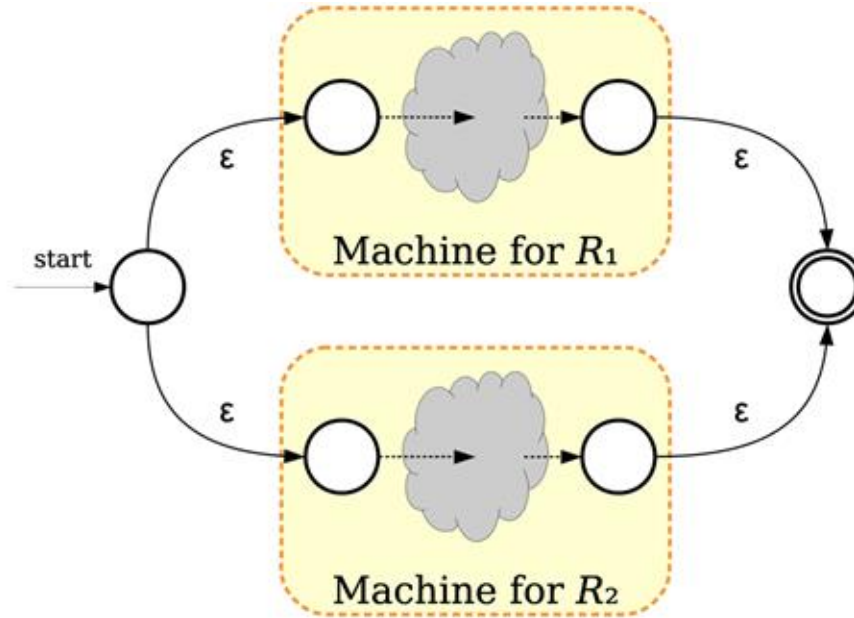
Automaton for single character **a**

Rules for Concatenation: R_1R_2



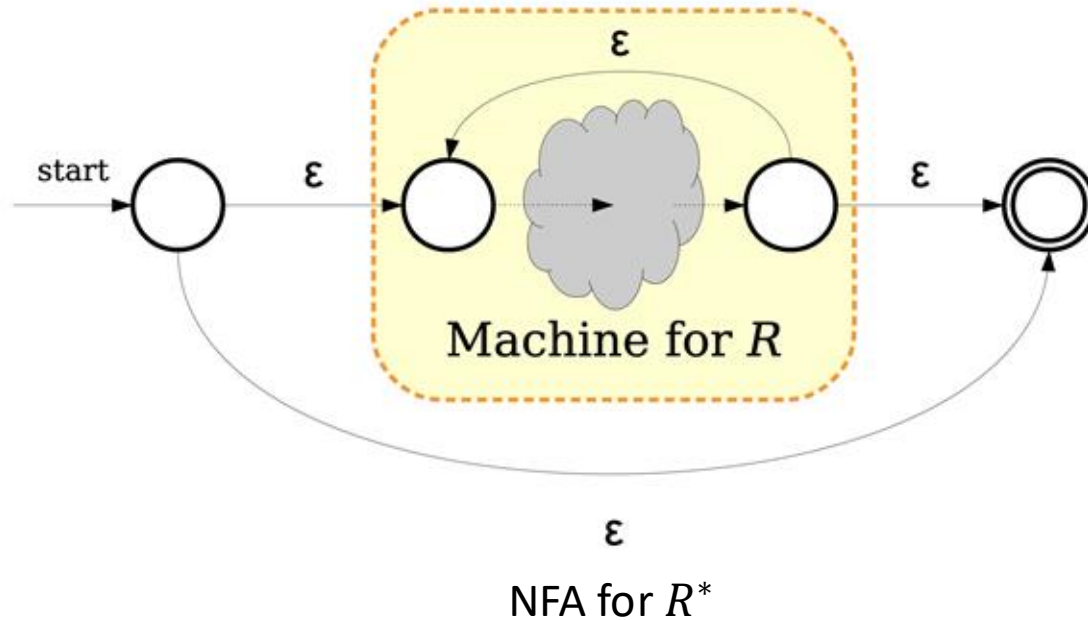
NFA for R_1R_2

Rules for Alternation: $R_1 | R_2$



NFA for $R_1 | R_2$

Rules for Kleene Star: R^*



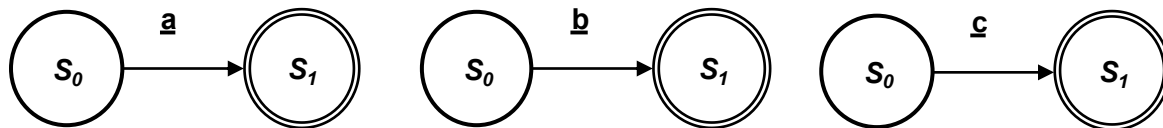
NFA Implementation

- To execute an NFA, we need to keep track of the set of current states
- By contrast, a DFA can only be in a single state at a time
- A natural compilation strategy from NFA $\rightarrow$ DFA:
 - The DFA has 2^n states for an NFA with n states
 - Each DFA state represents a subset of NFA states
 - Add transitions as specified in the NFA

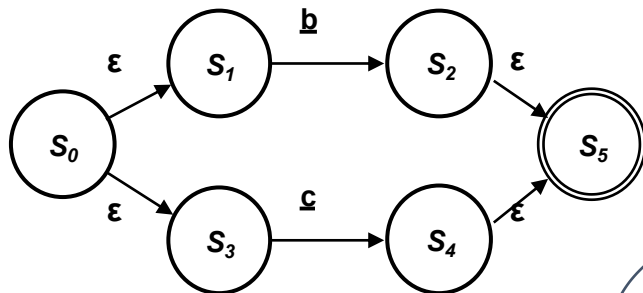
Example

RegEx: a (b | c)^{*}

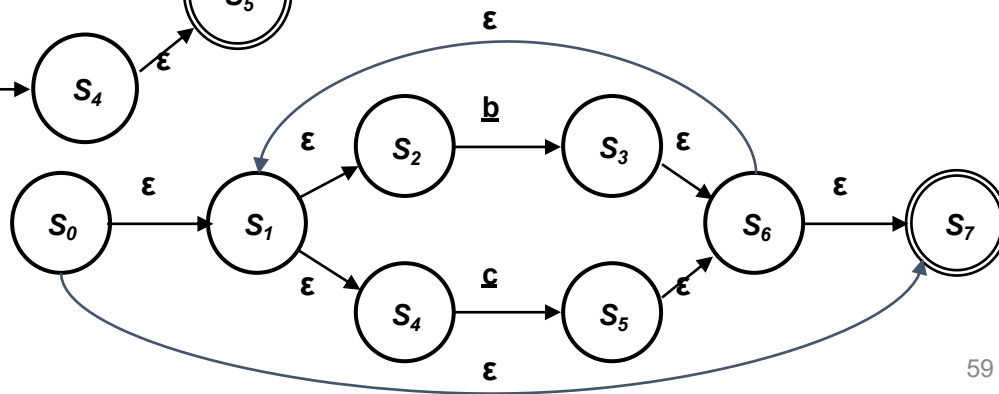
1. a, b, c



2. b | c

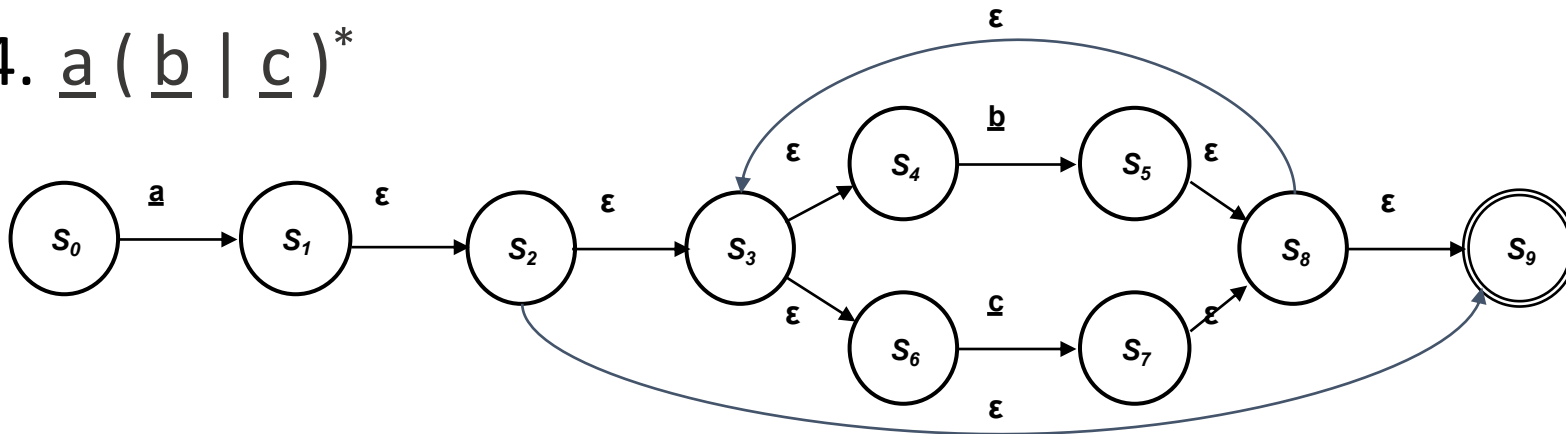


3. (b | c)^{*}



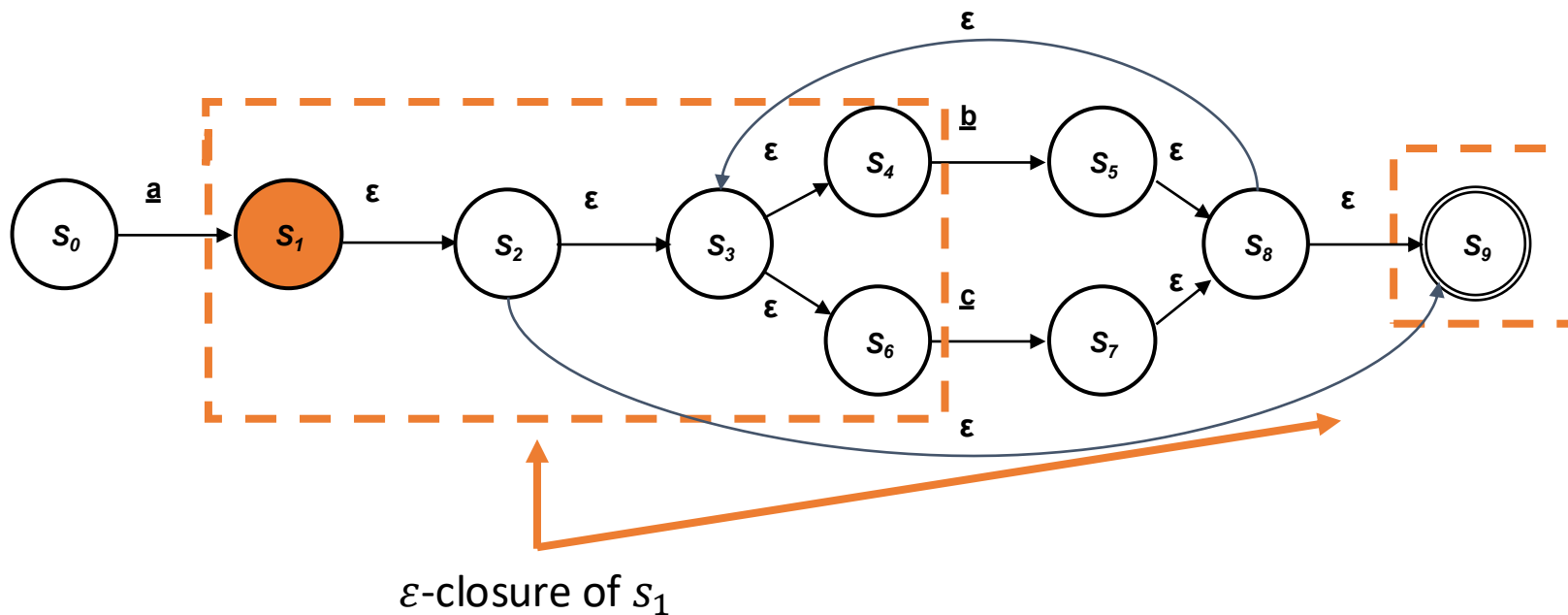
Example

4. $\underline{a}(\underline{b} \mid \underline{c})^*$

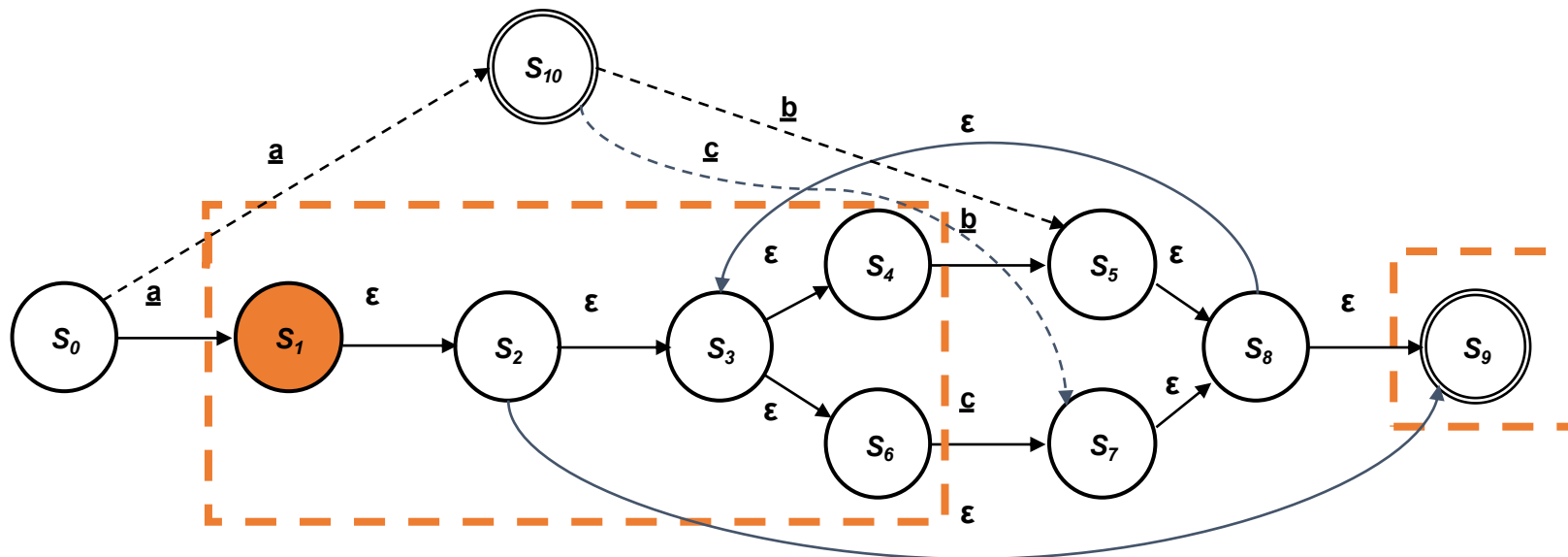


This is a little verbose...

Removing ϵ -transitions

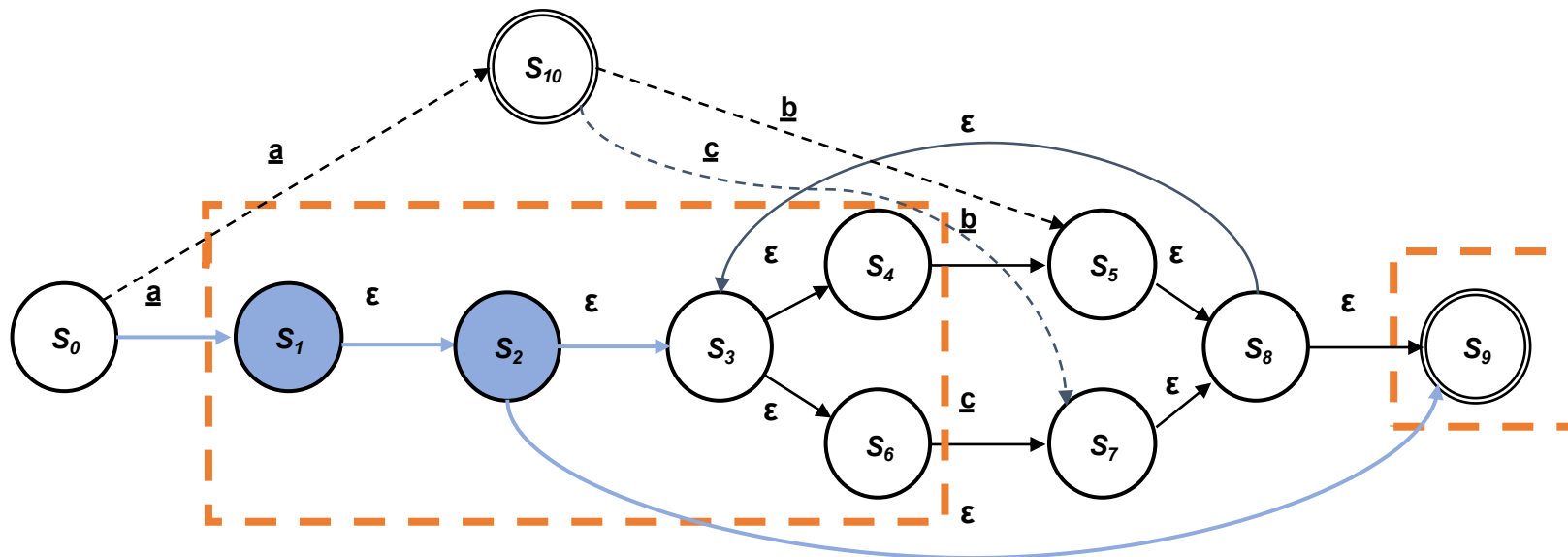


Removing ϵ -transitions



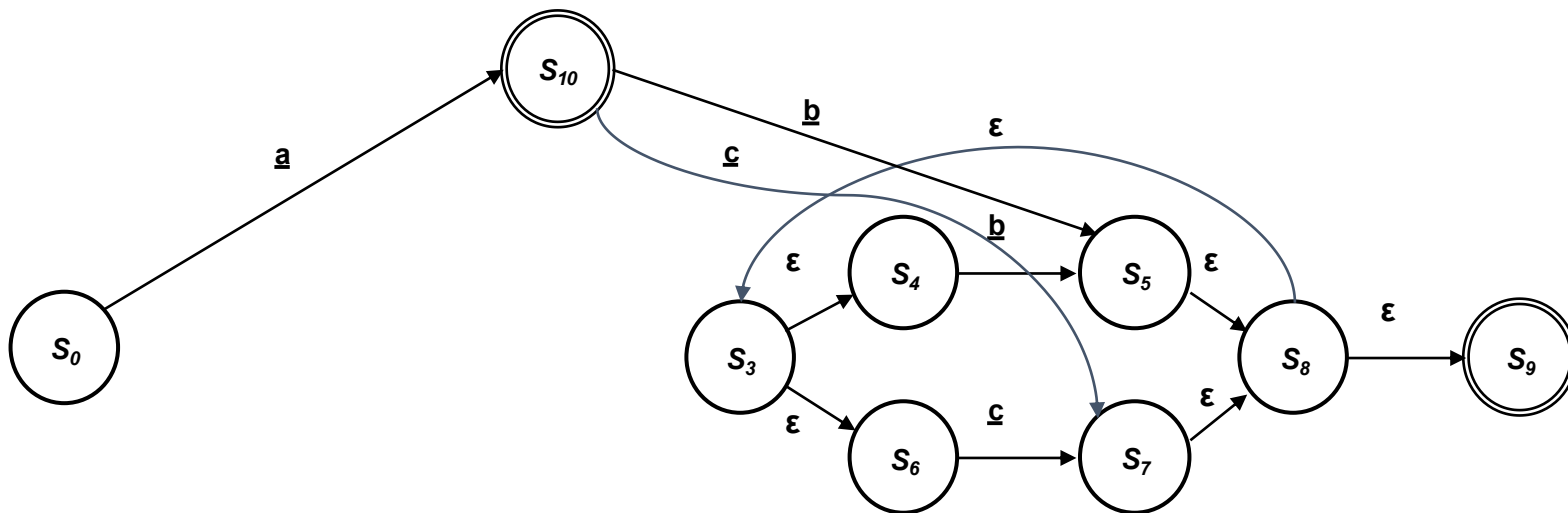
Create a new state $s_{10} = \{s_1, s_2, s_3, s_4, s_6, s_9\}$

Removing ϵ -transitions

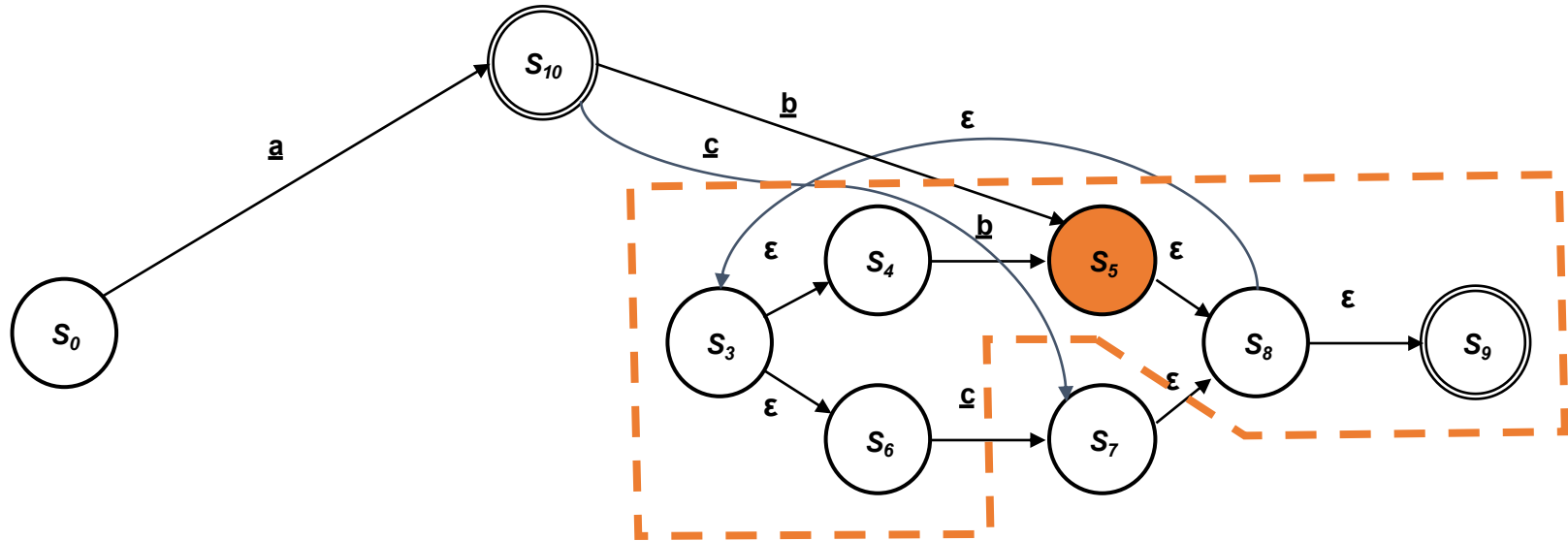


Remove s_1 and all states only reachable from s_1 , and associated transitions

Removing ϵ -transitions

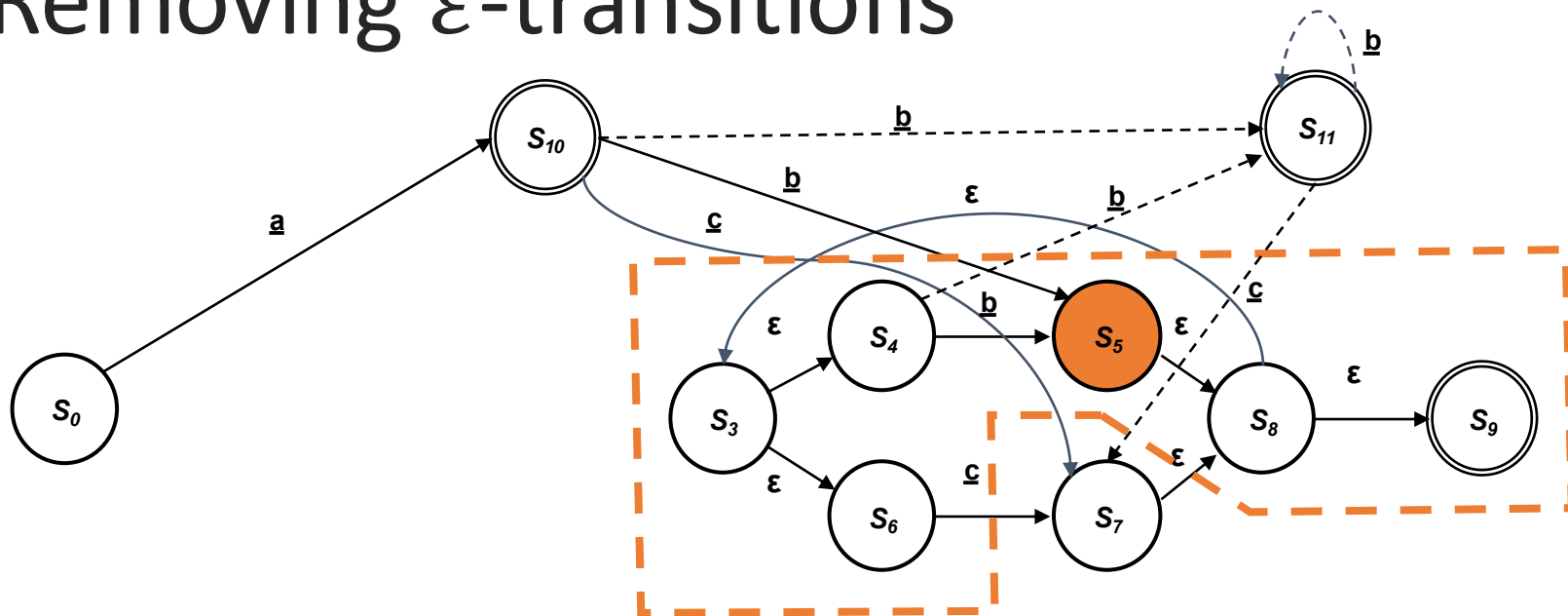


Removing ϵ -transitions

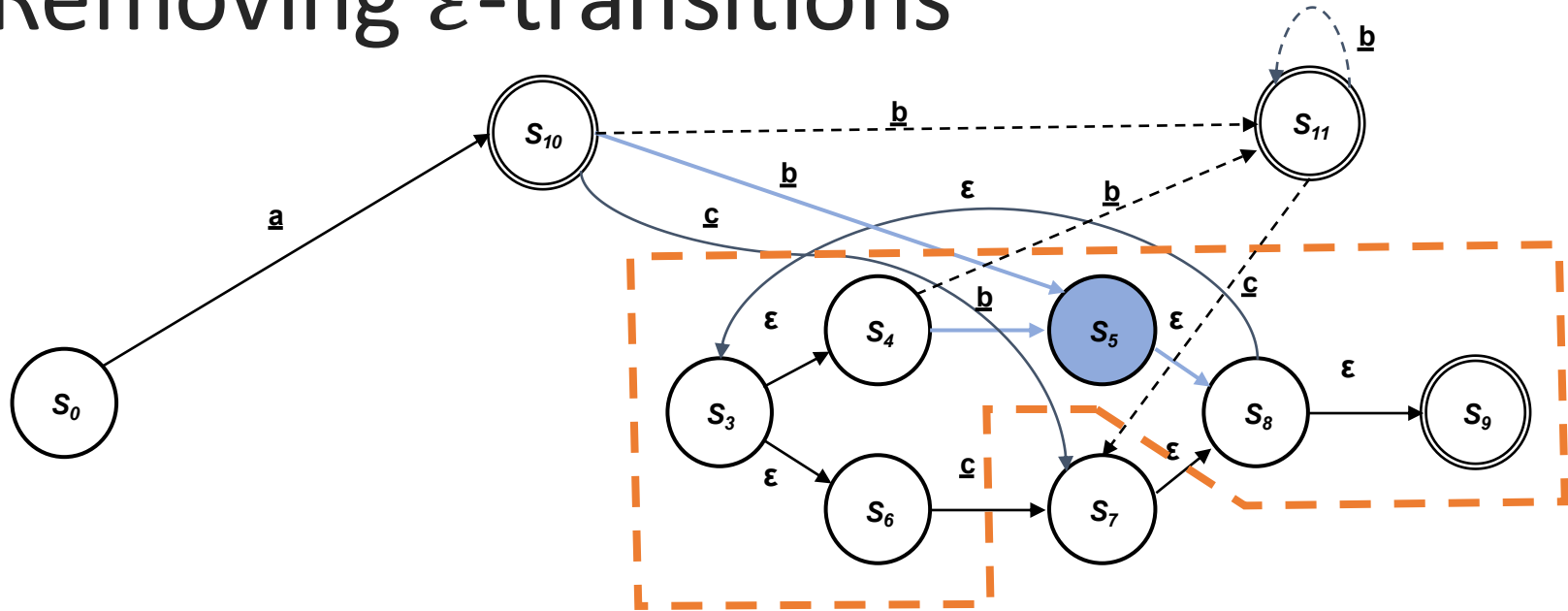


Repeat until all ϵ -transitions are removed

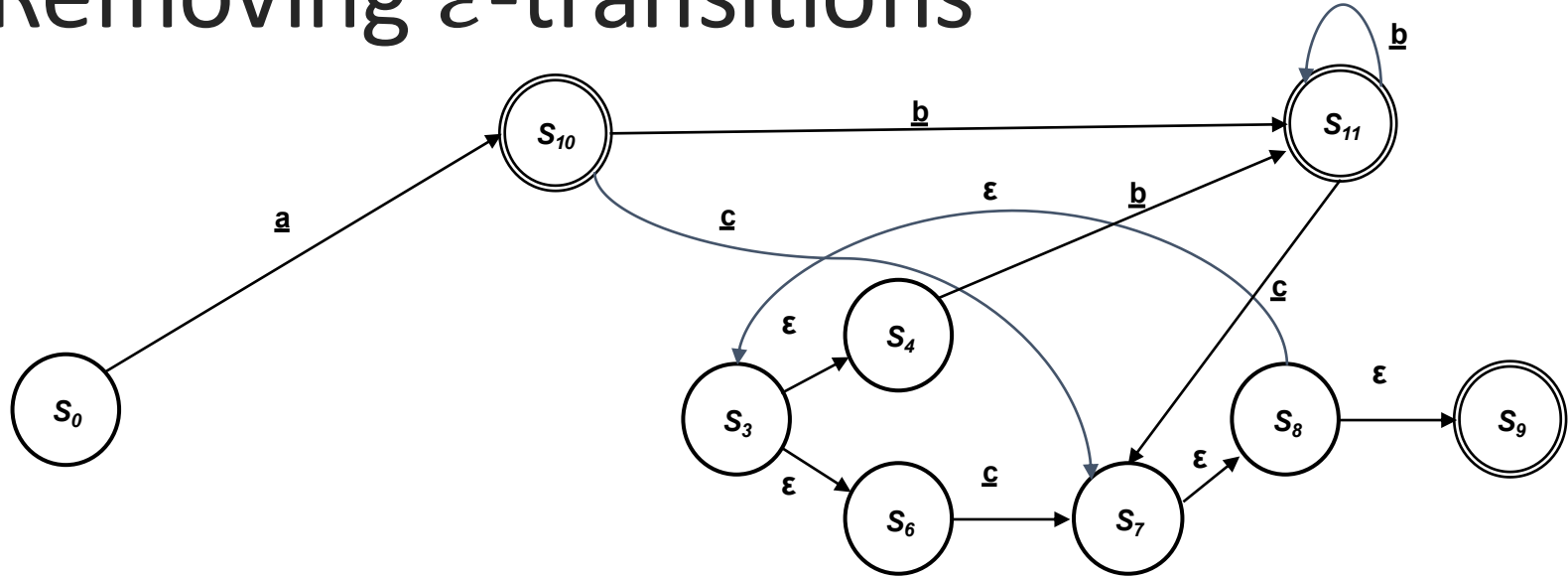
Removing ϵ -transitions



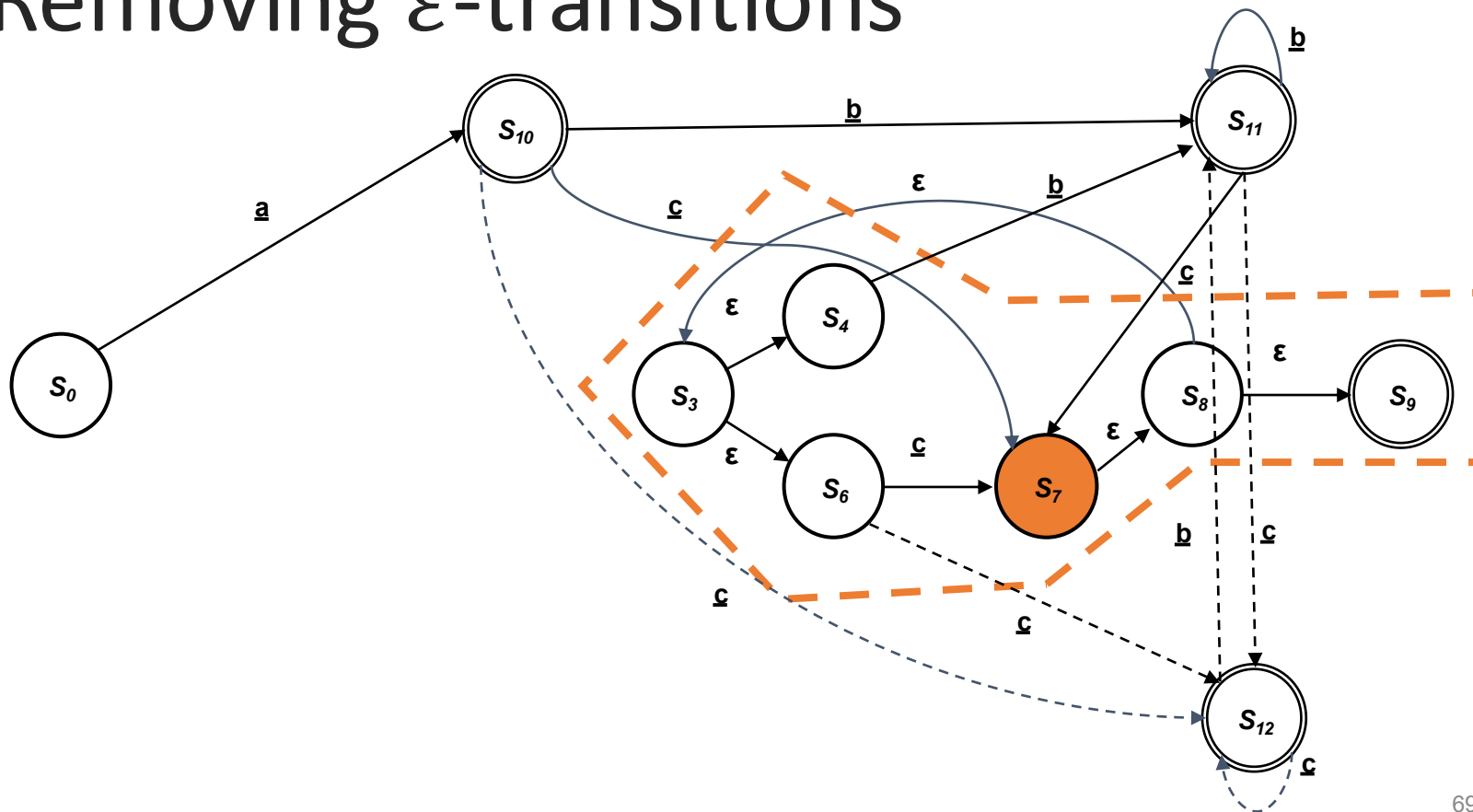
Removing ϵ -transitions



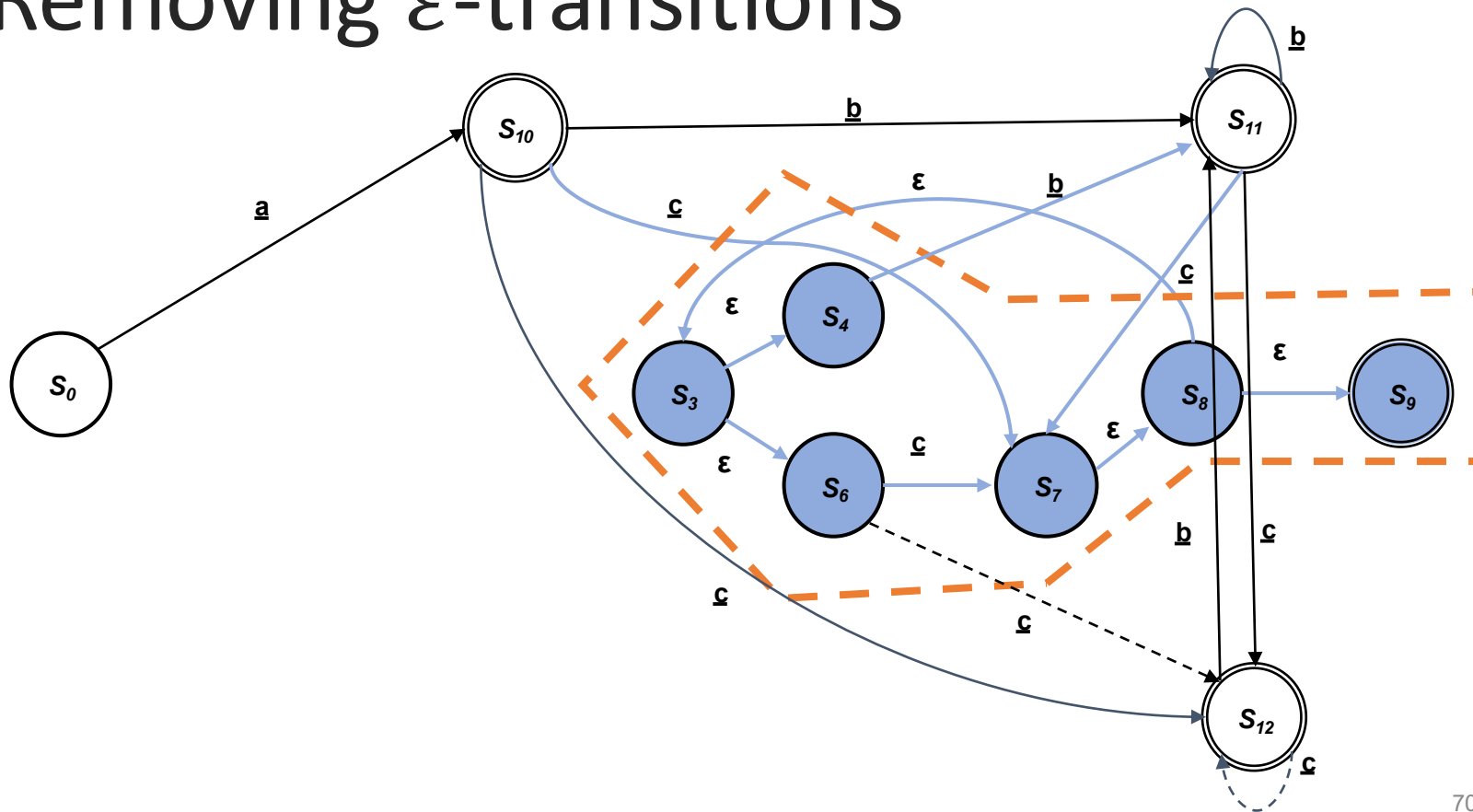
Removing ε -transitions



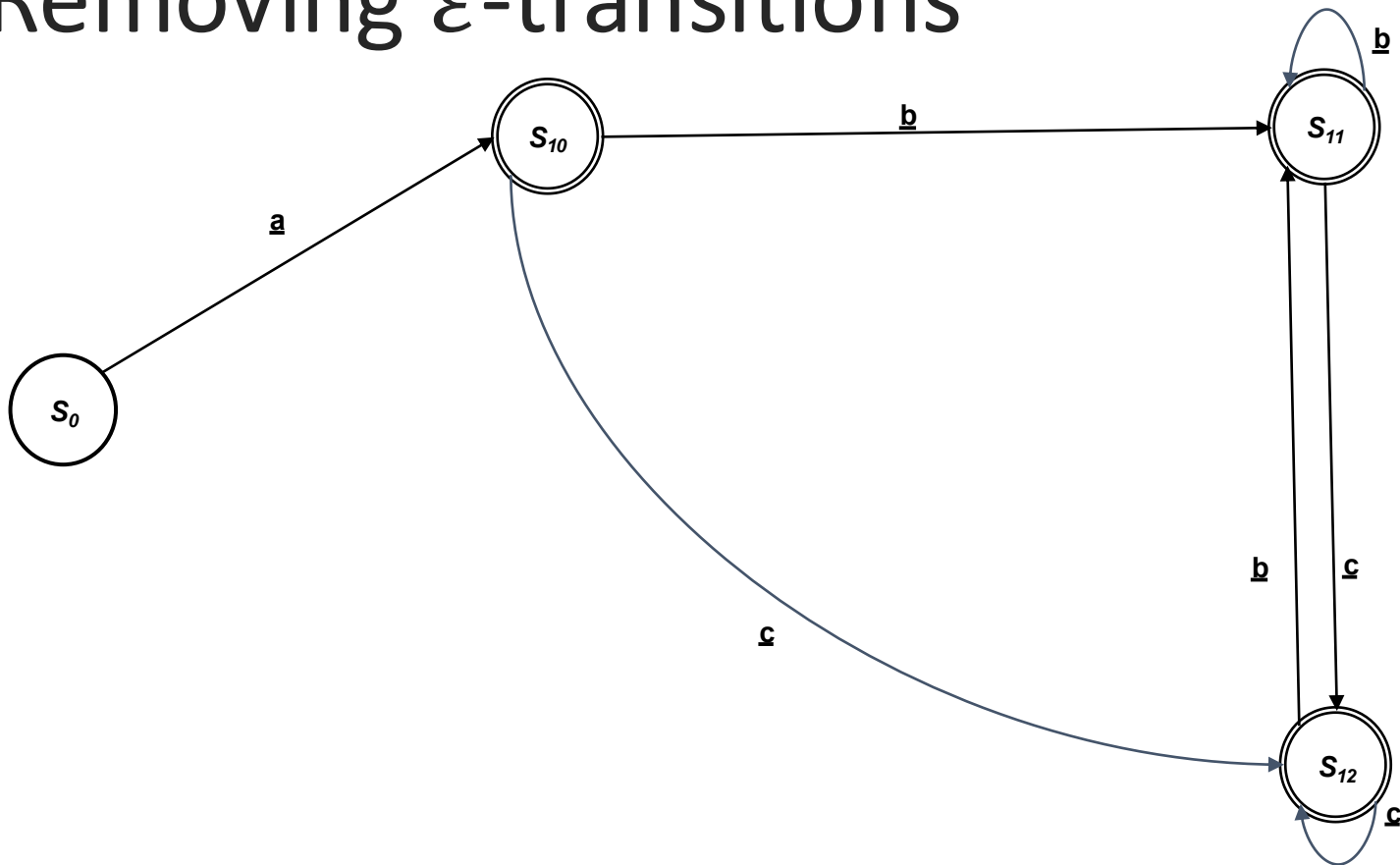
Removing ϵ -transitions



Removing ϵ -transitions

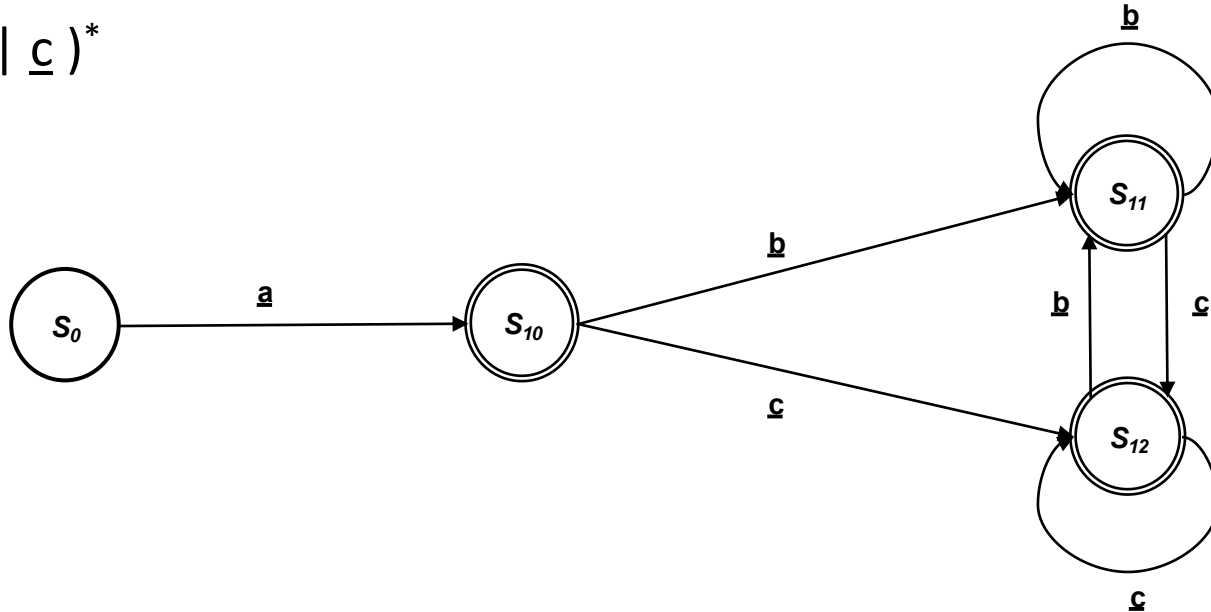


Removing ε -transitions

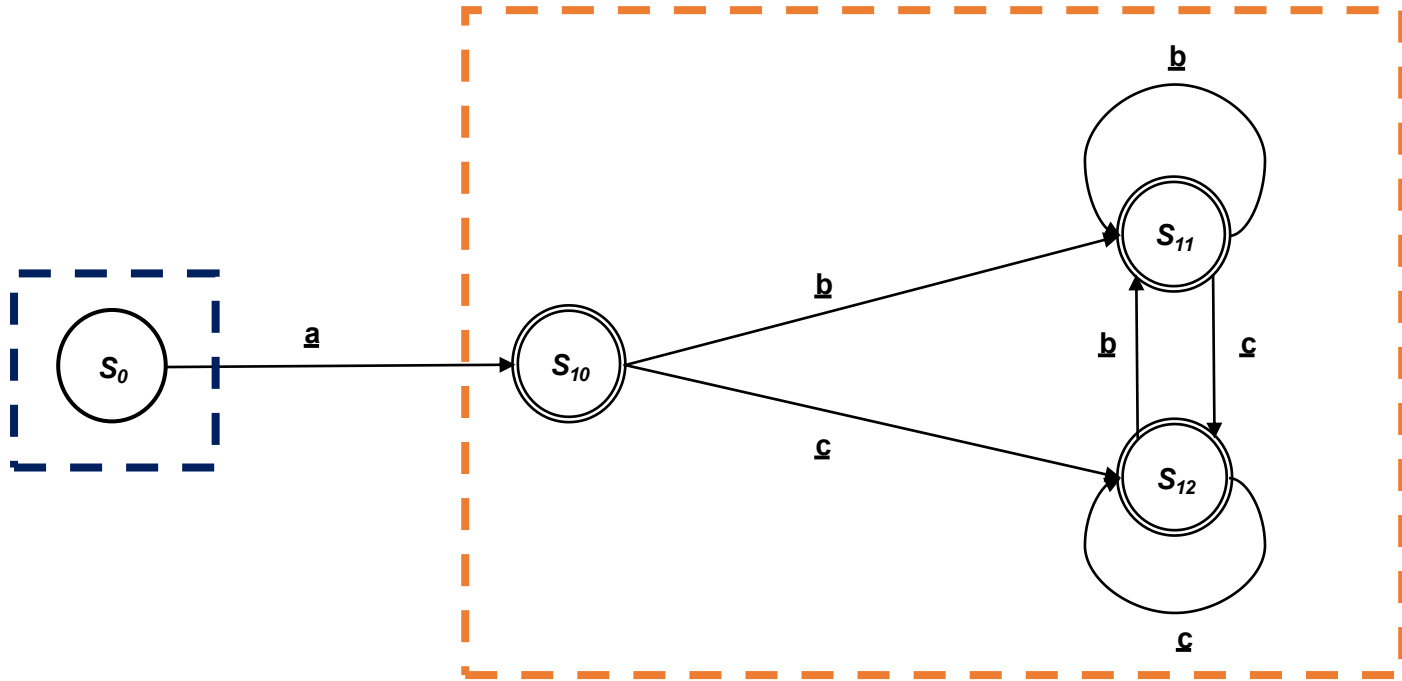


NFA → DFA

$\underline{a}(\underline{b} \mid \underline{c})^*$

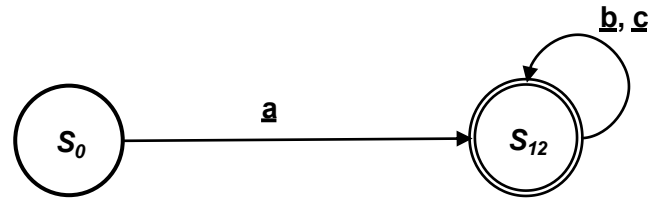


Minimizing DFA



Group states into 2 equivalent classes: final states and non-final states

Minimizing DFA



Split the equivalent classes into smaller classes to remove non-determinism

Scanning with RE

A scanner takes an ordered list of REs. For each RE, read one character at a time, and

- Output a token, or
- Wait to see next character, or
- Use next RE / report an error

Scanner Design & Implementation

- Recognizing multiple kinds of token – different terminal states
 - The scanner returns the type of token and when needed also the character-string of the token (for identifiers/literals)
- Longest possible token – if some tokens are prefixes of other tokens
 - Usually, keywords are treated as exceptions to identifiers
 - Buffering of the shorter token matches in case the longer token resulted in an error



Acknowledgements

- Keith Schwarz, Stanford CS103 slides
- Keith D. Cooper and Linda Torczon, Rice COMP412 slides
- Eric Atkinson, Binghamton CS471 slides